

Лабораторная работа 3

Ковалевский Климентий Евгеньевич, группа 7

1. Постановка задачи

План города представляет собой неориентированный граф, где вершины — это перекрёстки, а рёбра — дороги. Каждая дорога характеризуется временем движения по ней.

Особенность задачи: Помимо времени движения по дорогам, необходимо учитывать время на преодоление самого перекрёстка. Время преодоления перекрёстка i рассчитывается как $q \cdot k_i$, где:

- q — заданная константа;
- k_i — количество дорог, подходящих к перекрёстку i (степень вершины).

При этом конечный перекрёсток f преодолевать не нужно (время на него не тратится), а стартовый перекрёсток s преодолевается как обычный перекрёсток (то есть при выезде из s время $q \cdot k_s$ учитывается).

Цель: Найти кратчайший по времени маршрут от перекрёстка s до перекрёстка f .

Формат входных данных:

- Первая строка: N (количество перекрёстков, $1 \leq N \leq 11000$) и M (количество дорог, $0 \leq M \leq 100000$).
- Следующие M строк: u, v, w — номера соединяемых перекрёстков и время движения по дороге ($0 \leq w \leq 1000$).
- Последняя строка: s, f, q — стартовый перекрёсток, конечный перекрёсток и константа ($1 \leq q \leq 100$).

Формат выходных данных:

- Если маршрут существует: в первой строке **Yes**, во второй — минимальное время.
- Если маршрут не существует: вывести **No**.

2. Математическая модель

Пусть дан граф $G = (V, E)$, где V — множество вершин (перекрёстков), E — множество рёбер (дорог).

- k_u — степень вершины u (количество инцидентных рёбер).
- $w(u, v)$ — вес ребра (время движения по дороге между u и v).

Стоимость перехода из вершины u в смежную вершину v равна:

$$Cost(u, v) = w(u, v) + q \cdot k_u$$

Необходимо найти путь $P = \{s = v_0, v_1, \dots, v_k = f\}$, минимизирующий сумму:

$$\sum_{i=0}^{k-1} (w(v_i, v_{i+1}) + q \cdot k_{v_i})$$

Так как все веса неотрицательны ($w \geq 0, q \geq 1, k_u \geq 0$), для решения задачи применим **алгоритм Дейкстры**.

3. Описание алгоритма решения

1. **Инициализация:** Считываем граф, строим списки смежности и вычисляем степени всех вершин k_i . Инициализируем массив кратчайших расстояний **dist** бесконечностью, **dist[s] = 0**. Создаем приоритетную очередь (мин-кучу) и помещаем туда стартовую вершину.
2. **Основной цикл:**
 - Извлекаем вершину u с минимальным текущим расстоянием.
 - Если расстояние устарело, пропускаем.
 - Если $u = f$, прерываем цикл (так как мы не должны добавлять время на преодоление конечного перекрёстка).
 - Для каждого соседа v вершины u :
 - Вычисляем новое расстояние: **new_dist = dist[u] + w(u, v) + q * degree[u]**.
 - Если **new_dist < dist[v]**, обновляем **dist[v]** и добавляем v в очередь.
3. **Вывод результата:** Если **dist[f]** осталось бесконечным, выводим **No**.
Иначе выводим **Yes** и значение **dist[f]**.

4. Реализация программы (Python)

```
import heapq

N, M = map(int, input().split())

graph = [[] for _ in range(N + 1)]
degree = [0] * (N + 1)

for _ in range(M):
    u, v, w = map(int, input().split())

    graph[u].append((v, w))
    graph[v].append((u, w))

    degree[u] += 1
    degree[v] += 1

s, f, q = map(int, input().split())

INF = 10**30

dist = [INF] * (N + 1)
dist[s] = 0

heap = [(0, s)]

while heap:
    cur_dist, u = heapq.heappop(heap)

    if cur_dist != dist[u]:
        continue
    if u == f:
        break

    for v, road_time in graph[u]:
        # Время перехода = время в пути + время на преодоление текущего
        # перекрестка u
        new_dist = cur_dist + road_time + q * degree[u]

        if new_dist < dist[v]:
            dist[v] = new_dist
            heapq.heappush(heap, (new_dist, v))

if dist[f] == INF:
    print("No")
else:
    print("Yes")
    print(dist[f])
```

5. Разбор примера из условия

Входные данные:

```
6 9
1 2 2
1 3 1
2 4 1
2 5 1
3 4 3
3 5 1
4 5 2
4 6 1
5 6 1
1 6 1
```

- $N = 6, M = 9, s = 1, f = 6, q = 1$.
- Вычислим степени вершин: $\deg(1) = 2, \deg(2) = 3, \deg(3) = 3, \deg(4) = 4, \deg(5) = 4, \deg(6) = 2$.
- Рассмотрим путь $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$:
 1. Выезд из 1 в 3: время = $w(1, 3) + q \cdot \deg(1) = 1 + 1 \cdot 2 = 3$.
 2. Выезд из 3 в 5: время = $w(3, 5) + q \cdot \deg(3) = 1 + 1 \cdot 3 = 4$.
 3. Выезд из 5 в 6: время = $w(5, 6) + q \cdot \deg(5) = 1 + 1 \cdot 4 = 5$.
 4. Прибытие в 6. Время на преодоление 6 не тратится.
- Итоговое время: $3 + 4 + 5 = 12$.
- Программа выведет **Yes** и **12**, что совпадает с ожидаемым ответом.

6. Оценка сложности

- **Временная сложность:** $O((N + M) \log N)$. Алгоритм Дейкстры с использованием бинарной кучи требует $O(M \log N)$ времени на обработку рёбер и $O(N \log N)$ на извлечение вершин. При $N \leq 11000$ и $M \leq 100000$ данная сложность укладывается в лимиты времени (обычно 1-2 секунды).
- **Пространственная сложность:** $O(N + M)$. Хранение списков смежности требует $O(N + M)$ памяти, массивы **dist** и **degree** занимают $O(N)$.

7. Заключение

В ходе лабораторной работы была решена задача поиска кратчайшего пути в графе с нестандартной весовой функцией, учитывающей штраф за проезд через

перекрёстки. Модификация алгоритма Дейкстры (добавление $q \cdot degree[u]$ при релаксации рёбер) позволила корректно учесть все условия задачи. Программа успешно прошла проверку на примере из условия.